



Tecnicatura Universitaria en Programación Base de Datos II

Equipo # 60

Integrantes:

**SEGOVIA, ARY SEBASTIAN - 18655
SIROVICA, MIRKO SALDAÑO- 31134**

Explicación del Sistema

La base de datos **sistemaHoteleria** brinda servicio a un sistema de gestión para un hotel. Permite administrar:

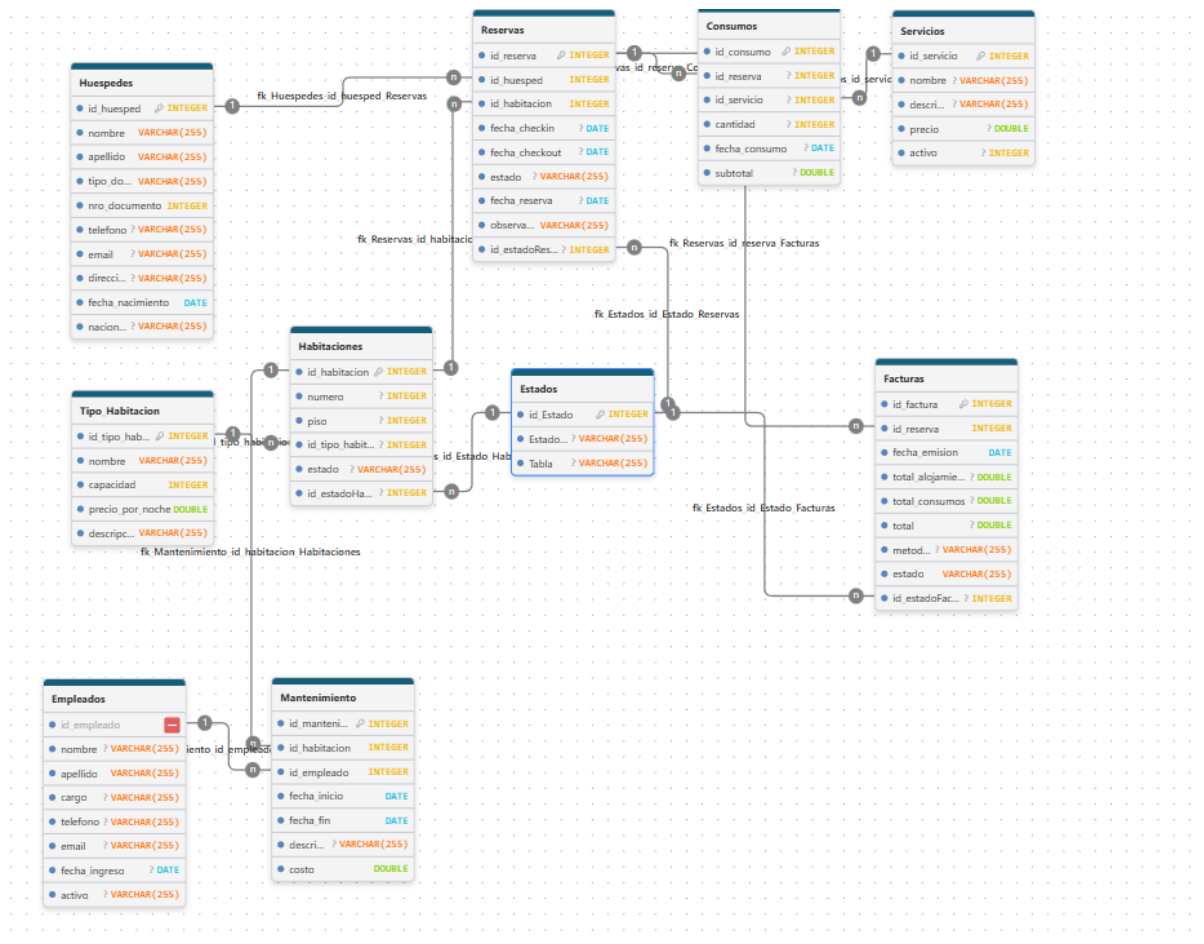
- Huéspedes: registro de datos personales (documento, contacto, nacionalidad).
- Habitaciones: 50 habitaciones distribuidas en 5 pisos con distintos tipos (Single, Doble, Triple, Suite).
- Reservas: gestión del ciclo completo de una reserva (Pendiente → Confirmada → En curso → Finalizada/Cancelada)
- Consumos: servicios adicionales consumidos durante la estadía (minibar, spa, restaurante, etc.)
- Facturación: generación de facturas al finalizar una estadía con totales de alojamiento y consumos.
- Mantenimiento: registro de trabajos de mantenimiento con asignación de empleados
- Empleados: personal del hotel con cargo y estado activo/inactivo.

Tabla centralizada de Estados

Los estados de las entidades principales (Habitaciones, Reservas, Facturas) se gestionan a través de una tabla centralizada Estados. Cada entidad tiene un campo numérico (FK lógica) que referencia a esta tabla, y un campo texto que se popula automáticamente por triggers. Esto permite:

- Agregar nuevos estados sin modificar lógica.
- Consistencia en la nomenclatura de estados.

Diagrama de Entidad Relación



DER

Objetos de Base de Datos clave en el sistema

Funciones de usuario

Se utilizan dentro de vistas para encapsular lógica de cálculo compleja, evitando repetir código. Si el cálculo cambia, solo se modifica la función y todas las vistas que la usan se actualizan automáticamente.

fn_TotalEstimadoReserva

Recibe un id_reserva y devuelve el total estimado de la estadía (alojamiento + consumos). Calcula el precio por noche multiplicado por la cantidad de noches y le suma todos los consumos asociados.

```
CREATE FUNCTION fn_TotalEstimadoReserva(@id_reserva INT)
RETURNS DECIMAL(10,2)
AS
BEGIN
    DECLARE @total DECIMAL(10,2);
    SELECT @total =
        (r.PrecioHabitacion * DATEDIFF(DAY, r.fecha_checkin,
r.fecha_checkout))
        + ISNULL((SELECT SUM(c.subtotal) FROM Consumos c WHERE c.id_reserva
= r.id_reserva), 0)
    FROM Reservas r
    WHERE r.id_reserva = @id_reserva;
    RETURN ISNULL(@total, 0);
```

Fn_DiasHastaCheckOut

recibe un id_reserva y devuelve los días que faltan hasta el checkout. Permite saber de un vistazo cuanto le quedo de estadía a cada huésped.

```
CREATE FUNCTION fn_DiasHastaCheckout(@id_reserva INT)
RETURNS INT
AS
BEGIN
    DECLARE @dias INT;
    SELECT @dias = DATEDIFF(DAY, GETDATE(), r.fecha_checkout)
    FROM Reservas r
    WHERE r.id_reserva = @id_reserva;
    RETURN ISNULL(@dias, 0);
END;
```

fn_CantidadReservasHuesped

Recibe un id_huesped y devuelve la cantidad de reservas activas (no canceladas). Se usa para identificar huéspedes frecuentes.

```
CREATE FUNCTION fn_CantidadReservasHuesped(@id_huesped INT)
RETURNS INT
AS
BEGIN
    DECLARE @cantidad INT;
    SELECT @cantidad = COUNT(*)
    FROM Reservas
    WHERE id_huesped = @id_huesped AND estado <> 'Cancelada';
    RETURN ISNULL(@cantidad, 0);
END
```

Vistas

vw_Habitaciones_Detalle

Muestra todas las habitaciones con su tipo, capacidad y precio por noche.

```
CREATE VIEW vw_Habitaciones_Detalle AS
SELECT
    h.id_habitacion,
    h.numero,
    h.piso,
    th.nombre AS tipo_habitacion,
    th.capacidad,
    th.precio_por_noche,
    h.estado
FROM Habitaciones h
INNER JOIN Tipo_Habitacion th ON th.id_tipo_habitacion = h.id_tipo_habitacion;
```

vw_Reservas_Completas

Muestra las reservas con datos del huésped, habitación, tipo, cantidad de noches y observaciones.

```
CREATE VIEW vw_Reservas_Completas AS
SELECT
    r.id_reserva,
    hu.nombre + ' ' + hu.apellido AS huesped,
    hu.nro_documento,
    hu.telefono AS telefono_huesped,
    hab.numero AS numero_habitacion,
    hab.piso,
    th.nombre AS tipo_habitacion,
    th.precio_por_noche,
    r.fecha_checkin,
    r.fecha_checkout,
    DATEDIFF(DAY, r.fecha_checkin, r.fecha_checkout) AS noches,
    r.estado,
    r.fecha_reserva,
    r.observaciones
FROM Reservas r
INNER JOIN Huespedes hu ON hu.id_huesped = r.id_huesped
INNER JOIN Habitaciones hab ON hab.id_habitacion = r.id_habitacion
INNER JOIN Tipo_Habitacion th ON th.id_tipo_habitacion = hab.id_tipo_habitacion;
```

vw_Habitaciones_Disponibles

Listado de habitaciones libres con tipo, capacidad y precio.

```
CREATE VIEW vw_Habitaciones_Disponibles AS
SELECT
    h.numero,
    h.piso,
    th.nombre AS tipo_habitacion,
    th.capacidad,
    th.precio_por_noche
FROM Habitaciones h
INNER JOIN Tipo_Habitacion th ON th.id_tipo_habitacion = h.id_tipo_habitacion
WHERE h.estado = 'Disponible';
```

vw_Mantenimientos_Activos

Habitaciones actualmente en mantenimiento con empleado asignado y días transcurridos.

```
CREATE VIEW vw_Mantenimientos_Activos AS
SELECT
    m.id_mantenimiento,
    hab.numero AS numero_habitacion,
    hab.piso,
    e.nombre + ' ' + e.apellido AS empleado_asignado,
    e.cargo,
    m.fecha_inicio,
    DATEDIFF(DAY, m.fecha_inicio, GETDATE()) AS dias_en_mantenimiento,
    m.descripcion,
    m.costo
FROM Mantenimiento m
INNER JOIN Habitaciones hab ON hab.id_habitacion = m.id_habitacion
INNER JOIN Empleados e ON e.id_empleado = m.id_empleado
WHERE m.fecha_fin IS NULL;
```

vw_Reservas_ConTotalEstimado (usa funciones de usuario)

Reservas activas con total estimado y días restantes. Utiliza las funciones fn_TotalEstimadoReserva y fn_DiasHastaCheckout para calcular estos valores de forma dinámica.

```
CREATE VIEW vw_Reservas_ConTotalEstimado AS
SELECT
    r.id_reserva,
    hu.nombre + ' ' + hu.apellido AS huesped,
    hab.numero AS numero_habitacion,
    th.nombre AS tipo_habitacion,
    r.fecha_checkin,
    r.fecha_checkout,
    r.estado,
    dbo.fn_TotalEstimadoReserva(r.id_reserva) AS total_estimado,
    dbo.fn_DiasHastaCheckout(r.id_reserva) AS dias_restantes
FROM Reservas r
INNER JOIN Huespedes hu ON hu.id_huesped = r.id_huesped
INNER JOIN Habitaciones hab ON hab.id_habitacion = r.id_habitacion
INNER JOIN Tipo_Habitacion th ON th.id_tipo_habitacion = hab.id_tipo_habitacion
WHERE r.estado IN ('En Curso', 'Confirmada', 'Pendiente');
```

vw_Huespedes_Frecuentes (usa funciones de usuario)

Huéspedes con al menos una reserva activa, mostrando cantidad de reservas y gasto total. Utiliza la función fn_CantidadReservasHuesped tanto para filtrar como para mostrar la cantidad.

```
CREATE VIEW vw_Huespedes_Frecuentes AS

SELECT
    hu.id_huesped,
    hu.nombre + ' ' + hu.apellido AS huesped,
    hu.nro_documento,
    hu.nacionalidad,
    dbo.fn_CantidadReservasHuesped(hu.id_huesped) AS reservas_activas,
    (SELECT COUNT(*) FROM Reservas r WHERE r.id_huesped = hu.id_huesped) AS
    reservas_totales,
    ISNULL((SELECT SUM(f.total) FROM Facturas f INNER JOIN Reservas r ON r.id_reserva =
    f.id_reserva WHERE r.id_huesped = hu.id_huesped AND f.estado = 'Pagada'), 0) AS
    gasto_total
FROM Huespedes hu
WHERE dbo.fn_CantidadReservasHuesped(hu.id_huesped) > 0;
```

Triggers tras insercion

trg_reserva_insert

Al insertar una reserva con estado En Curso (Id_EstadoReserva=6), la habitación asociada pasa automáticamente a Ocupada.

```
CREATE TRIGGER trg_reserva_insert
ON Reservas
AFTER INSERT
AS
BEGIN
    UPDATE Habitaciones
    SET id_estadoHabitacion = 2
    WHERE id_habitacion IN (
        SELECT id_habitacion FROM inserted WHERE Id_EstadoReserva = 6
    );
END;
```


trg_consumo_insert

Al insertar un consumo, calcula automáticamente el subtotal multiplicando la cantidad por el precio del servicio.

```
CREATE TRIGGER trg_consumo_insert
ON Consumos
AFTER INSERT
AS
BEGIN
    UPDATE c
    SET c.subtotal = i.cantidad * s.precio
    FROM Consumos c
    INNER JOIN inserted i ON i.id_consumo = c.id_consumo
    INNER JOIN Servicios s ON s.id_servicio = c.id_servicio;
END;
```

trg_mantenimiento_insert

Al insertar un mantenimiento sin fecha de fin (activo), la habitación pasa a En Mantenimiento.

```
CREATE TRIGGER trg_mantenimiento_insert
ON Mantenimiento
AFTER INSERT
AS
BEGIN
    UPDATE Habitaciones
    SET id_estadoHabitacion = 3
    WHERE id_habitacion IN (SELECT id_habitacion FROM inserted WHERE fecha_fin IS NULL);
END;
```

trg_reserva_insertaEstado

Al insertar una reserva, popula el campo estado (texto) buscando el nombre en la tabla Estados según Id_EstadoReserva.

```
CREATE TRIGGER trg_reserva_insertaEstado
ON Reservas
AFTER INSERT
AS
BEGIN
    UPDATE r
    SET r.estado = e.EstadoNombre
    FROM Reservas r
    INNER JOIN inserted i ON i.id_reserva = r.id_reserva
    INNER JOIN Estados e ON e.Id_Estado = r.Id_EstadoReserva
    WHERE e.Tabla = 'Reservas'
END;
```

trg_habitaciones_insertaEstado

Al insertar una habitación, popula el campo estado (texto) buscando el nombre en la tabla Estados según id_estadoHabitacion.

```
CREATE TRIGGER trg_habitaciones_insertaEstado
ON Habitaciones
AFTER INSERT
AS
BEGIN
    UPDATE h
    SET h.estado = e.EstadoNombre
    FROM Habitaciones h
    INNER JOIN inserted i ON i.id_habitacion = h.id_habitacion
    INNER JOIN Estados e ON e.Id_Estado = h.id_estadoHabitacion
    WHERE e.Tabla = 'Habitaciones'
END;
```

trg_reserva_updatePrHabitacion

Al insertar una reserva, llena automáticamente el campo PrecioHabitacion con el precio por noche del tipo de habitación reservada.

```
CREATE TRIGGER trg_reserva_updatePrHabitacion
ON Reservas
AFTER INSERT
AS
BEGIN
    UPDATE r
    SET r.PrecioHabitacion = th.precio_por_noche
    FROM Reservas r
    INNER JOIN inserted i ON i.id_reserva = r.id_reserva
    INNER JOIN Habitaciones h ON h.id_habitacion = r.id_habitacion
    INNER JOIN Tipo_Habitacion th ON th.id_tipo_habitacion = h.id_tipo_habitacion
END;
```

trg_facturas_insertaEstado

Al insertar una factura, popula el campo estado (texto) buscando el nombre en la tabla Estados según id_estadoFactura

```
CREATE TRIGGER trg_facturas_insertaEstado
ON Facturas
AFTER INSERT
AS
BEGIN
    UPDATE f
    SET f.estado = e.EstadoNombre
    FROM Facturas f
    INNER JOIN inserted i ON i.id_factura = f.id_factura
    INNER JOIN Estados e ON e.Id_Estado = f.id_estadoFactura
    WHERE e.Tabla = 'Facturas'
END;
```

Triggers tras eliminación

trg_consumo_delete

Al eliminar un consumo, recalcula los campos total_consumos y total en la factura asociada a esa reserva.

```
CREATE TRIGGER trg_consumo_delete
ON Consumos
AFTER DELETE
AS
BEGIN
    UPDATE f
    SET f.total_consumos = ISNULL((
        SELECT SUM(c.subtotal)
        FROM Consumos c
        WHERE c.id_reserva = f.id_reserva
    ), 0),
    f.total = f.total_alojamiento + ISNULL((
        SELECT SUM(c.subtotal)
        FROM Consumos c
        WHERE c.id_reserva = f.id_reserva
    ), 0)
    FROM Facturas f
    INNER JOIN deleted d ON d.id_reserva = f.id_reserva;
END;
```

trg_reserva_delete

Al eliminar una reserva que estaba En Curso (Id_EstadoReserva=6), la habitación vuelve a Disponible.

```
CREATE TRIGGER trg_reserva_delete
ON Reservas
AFTER DELETE
AS
BEGIN
    UPDATE Habitaciones
    SET id_estadoHabitacion = 1
    WHERE id_habitacion IN (
        SELECT id_habitacion FROM deleted WHERE Id_EstadoReserva = 6);
END;
```

Triggers tras Modificacion

trg_reserva_update

Al cambiar el Id_EstadoReserva de una reserva: si pasa a Finalizada (7) o Cancelada (8), la habitación vuelve a Disponible. Si pasa a En Curso (6), la habitación se marca como Ocupada.

```
CREATE TRIGGER trg_reserva_update
ON Reservas
AFTER UPDATE
AS
BEGIN
    IF UPDATE(Id_EstadoReserva)
    BEGIN
        UPDATE Habitaciones
        SET id_estadoHabitacion = 1
        WHERE id_habitacion IN (
            SELECT id_habitacion FROM inserted WHERE Id_EstadoReserva IN (7, 8));

        UPDATE Habitaciones
        SET id_estadoHabitacion = 2
        WHERE id_habitacion IN (
            SELECT id_habitacion FROM inserted WHERE Id_EstadoReserva = 6);
    END;
END;
```

trg_mantenimiento_update

Cuando se carga la fecha de fin en un mantenimiento (se finaliza el trabajo), la habitación vuelve a Disponible.

```
CREATE TRIGGER trg_mantenimiento_update
ON Mantenimiento
AFTER UPDATE
AS
BEGIN
    IF UPDATE(fecha_fin)
    BEGIN
        UPDATE Habitaciones
        SET id_estadoHabitacion = 1
        WHERE id_habitacion IN (
            SELECT I.id_habitacion
            FROM inserted I
            INNER JOIN deleted D ON D.id_mantenimiento = I.id_mantenimiento
            WHERE D.fecha_fin IS NULL AND I.fecha_fin IS NOT NULL
        );
    END;
END;
```

trg_reserva_updateEstado

Al modificar Id_EstadoReserva en una reserva, actualiza el campo estado (texto) desde la tabla Estados.

```
CREATE TRIGGER trg_reserva_updateEstado
ON Reservas
AFTER UPDATE
AS
BEGIN
    IF UPDATE(Id_EstadoReserva)
    BEGIN
        UPDATE r
        SET r.estado = e.EstadoNombre
        FROM Reservas r
        INNER JOIN inserted i ON i.id_reserva = r.id_reserva
        INNER JOIN Estados e ON e.Id_Estado = r.Id_EstadoReserva
        WHERE e.Tabla = 'Reservas'
    END;
END;
```

trg_habitaciones_updateEstado

Al modificar id_estadoHabitacion en una habitación, actualiza el campo estado (texto) desde la tabla Estados.

```
CREATE TRIGGER trg_habitaciones_updateEstado
ON Habitaciones
AFTER UPDATE
AS
BEGIN
    UPDATE h
    SET h.estado = e.EstadoNombre
    FROM Habitaciones h
    INNER JOIN inserted i ON i.id_habitacion = h.id_habitacion
    INNER JOIN Estados e ON e.Id_Estado = h.id_estadoHabitacion
    WHERE e.Tabla = 'Habitaciones'
END;
```

trg_facturas_updateEstado

Al modificar id_estadoFactura en una factura, actualiza el campo estado (texto) desde la tabla Estados.

```
CREATE TRIGGER trg_facturas_updateEstado
ON Facturas
AFTER UPDATE
AS
BEGIN
    IF UPDATE(id_estadoFactura)
    BEGIN
        UPDATE f
        SET f.estado = e.EstadoNombre
        FROM Facturas f
        INNER JOIN inserted i ON i.id_factura = f.id_factura
        INNER JOIN Estados e ON e.Id_Estado = f.id_estadoFactura
        WHERE e.Tabla = 'Facturas'
    END;
END;
```

Procedimientos Almacenados

sp_CrearReserva

Crea una nueva reserva validando: existencia del huésped, existencia de la habitación, coherencia de fechas, disponibilidad de la habitación y ausencia de solapamiento con otras reservas.

```
ALTER PROCEDURE [dbo].[sp_CrearReserva]
    @id_huesped INT,
    @id_habitacion INT,
    @fecha_checkin DATE,
    @fecha_checkout DATE,
    @observaciones VARCHAR(255) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (
        SELECT 1
        FROM Huespedes
        WHERE id_huesped = @id_huesped
    )
    BEGIN
        RAISERROR('El huesped no existe.', 16, 1);
        RETURN;
    END;

    IF NOT EXISTS (
        SELECT 1
        FROM Habitaciones
        WHERE id_habitacion = @id_habitacion
    )
    BEGIN
        RAISERROR('La habitacion no existe.', 16, 1);
        RETURN;
    END;

    IF (@fecha_checkin >= @fecha_checkout)
    BEGIN
        RAISERROR('La fecha de check-out debe ser mayor a la fecha de check-in.', 16, 1);
        RETURN;
    END;

    IF EXISTS (
        SELECT 1
        FROM Habitaciones
        WHERE id_habitacion = @id_habitacion
        AND estado <> 'Disponible'
    )
    BEGIN
        RAISERROR('La habitacion no esta disponible.', 16, 1);
        RETURN;
    END;

    IF EXISTS (
        SELECT 1
        FROM Reservas
        WHERE id_habitacion = @id_habitacion
        AND estado <> 'Cancelada'
        AND @fecha_checkin < fecha_checkout
        AND @fecha_checkout > fecha_checkin
    )
    BEGIN
        RAISERROR('La habitacion ya tiene una reserva para esas fechas.', 16, 1);
        RETURN;
    END;

    INSERT INTO Reservas
    (
        id_huesped,
        id_habitacion,
        fecha_checkin,
        fecha_checkout,
        fecha_reserva,
        observaciones,
        Id_EstadoReserva
    )
    VALUES
    (
        @id_huesped,
        @id_habitacion,
        @fecha_checkin,
        @fecha_checkout,
        GETDATE(),
        @observaciones,
        4
    );

    PRINT 'Reserva creada correctamente.';
END;
```


sp_RegistrarCheckIn (Acción en la BD)

Registra el check-in de una reserva, validando que exista y que esté en estado Pendiente o Confirmada. Cambia el estado a En Curso, lo cual dispara triggers que actualizan la habitación a Ocupada.

```
CREATE PROCEDURE sp_RegistrarCheckIn(@id_reserva INT)
AS
BEGIN
    BEGIN TRY
        IF NOT EXISTS (SELECT 1 FROM Reservas WHERE id_reserva = @id_reserva)
        BEGIN
            RAISERROR('La reserva no existe.',16,1)
        END

        IF NOT EXISTS (
            SELECT 1 FROM Reservas
            WHERE id_reserva = @id_reserva
            AND (estado = 'Pendiente' OR estado = 'Confirmada')
        )
        BEGIN
            RAISERROR('La reserva no se encuentra en un estado válido para realizar el check-in.',16,1)
        END

        UPDATE Reservas SET Id_EstadoReserva = 6 WHERE id_reserva = @id_reserva

        PRINT 'Check-In realizado correctamente.'
    END TRY
    BEGIN CATCH
        PRINT ERROR_MESSAGE()
    END CATCH
END;
```

sp_ReporteReservasPorPeriodo

Devuelve un reporte de reservas filtrado por un rango de fechas, mostrando datos del huésped, habitación y tipo.

```
CREATE PROCEDURE sp_ReporteReservasPorPeriodo(@fechaInicio DATE, @fechaFin DATE)
AS
BEGIN
    BEGIN TRY
        IF @fechaInicio > @fechaFin
        BEGIN
            RAISERROR('La fecha de inicio no puede ser mayor que la fecha de fin.',16,1)
        END

        SELECT
            R.id_reserva, H.nombre, H.apellido,
            HA.numero AS Habitacion, TH.nombre AS TipoHabitacion,
            R.fecha_checkin, R.fecha_checkout, R.estado
        FROM Reservas R
        INNER JOIN Huespedes H ON R.id_huesped = H.id_huesped
        INNER JOIN Habitaciones HA ON R.id_habitacion = HA.id_habitacion
        INNER JOIN Tipo_Habitacion TH ON HA.id_tipo_habitacion = TH.id_tipo_habitacion
        WHERE R.fecha_checkin BETWEEN @fechaInicio AND @fechaFin
        ORDER BY R.fecha_checkin;
    END TRY
    BEGIN CATCH
        PRINT ERROR_MESSAGE()
    END CATCH
END;
```

Link a repositorio:

[GITHUB](#)

Link a video :

[Video](#)